CAPITAL GAINS · CLI · PYTHON

Refatorando um teste técnico: Lições de Python Avançado

Uma CLI para cálculo de imposto sobre ganhos de capital em operações de renda variável



QUEM SOU EU

Cássio Botaro

Engenheiro de Software · Época Cosméticos

CÓDIGO FONTE

O código está disponível

github.com/cassiobotaro/capital-gains

CONTEXTO

O Problema

ENTRADA

```
// stdin — lotes de operações em JSON
[
  { "operation": "buy",  "unit-cost": 10.00, "quantity": 10000 },
  { "operation": "sell", "unit-cost": 20.00, "quantity":  5000 },
  { "operation": "sell", "unit-cost":  5.00, "quantity":  5000 }
]
```

SAÍDA

```
// stdout — imposto por operação
[
  { "tax": 0.00    },  // compra → isento
  { "tax": 10000.00 },  // lucro tributável
  { "tax": 0.00    }   // prejuízo
]
```

- → **20%**
  sobre o lucro acima do preço médio ponderado
- → **Isenção**
  operações ≤ R$ 20.000,00
- → **Prejuízos**
  deduzidos de lucros futuros

VISÃO GERAL

A Solução

- **Uma CLI que lê lotes de operações via stdin**
  Cada linha da entrada é um array JSON de operações independentes, processadas sequencialmente
- **Estado acumulado por lote**
  Preço médio ponderado e prejuízo acumulado são mantidos entre operações do mesmo lote
- **Resultado auditável por operação**
  Cada operação retorna seu imposto e o novo estado, o que permite conferir cada passo
- **Objetos imutáveis do início ao fim**
  Nenhuma variável é mutada. Cada passo cria um novo estado a partir do anterior

Fórmula central

```
preço_médio_ponderado_novo = (
  (quantidade × preço_médio_ponderado) +
(quantidade_nova × preço_novo)
) / (quantidade + quantidade_nova)
```

Regra de imposto

```
lucro_líquido = max(0, lucro - prejuízo)
imposto = 20% × lucro_líquido
            se volume > R$ 20.000
```

ARQUITETURA

Três módulos, uma narrativa

cli

Leitura de stdin

Parse JSON

Escrita em stdout

→

money

Objeto de valor imutável

Aritmética monetária

Precisão decimal garantida

→

tax

Estado do investimento

Regras de negócio

Resultado auditável

MÓDULO 1 · CLI.PY / __MAIN__.PY

CLI — entrada e saída

__MAIN__.PY — PONTO DE ENTRADA

```python
from .cli import main

main()

# main() → run(sys.stdin, sys.stdout)
# sys.stdin  → Iterable[str]
# sys.stdout → Writer[str]
```

O contrato

`reader_stream` — qualquer `Iterable[str]`

`writer_stream` — qualquer `Writer[str]`

stdin/stdout em produção · qualquer objeto com `write()`

CLI.PY — RUN E MAIN

```python
def run(
    reader_stream: Iterable[str],
    writer_stream: Writer[str],
) -> None:
    for line in readlines(reader_stream):
        dump_json(
            process_operations_batch(
                parse_json_line(line)
            ),
            writer_stream,
        )

def main() -> None:
    run(sys.stdin, sys.stdout)
```

↓ expandido nos próximos slides

`readlines()` `parse_json_line()` `dump_json()`

CLI — as funções internas

```python
def readlines(reader: Iterable[str]) -> Iterator[str]:
    for line in reader:
        if line.strip():
            yield line


def parse_json_line(line: str) -> list[Operation]:
    raw_ops: list[RawOperation] = json.loads(line)
    return [
        Operation(
            kind=raw["operation"],
            unit_cost=Money(Decimal(str(raw["unit-cost"]))),
            quantity=raw["quantity"],
        )
        for raw in raw_ops
    ]


def dump_json(taxes: Iterable[OperationResult], output: Writer[str]) -> None:
    formatted = [{"tax": float(r.tax.amount)} for r in taxes]
    json.dump(formatted, output)
    output.write("\n")
```

01 Iterable[str] como parâmetro

mais simples que IO, aceita stdin, lista ou qualquer iterável

02 Iterator como retorno de readlines

`Iterator` exige `__next__`, mantém estado e só percorre uma vez

03 json.dump vs json.dumps

dump escreve direto no stream, sem montar a string em memória

04 `Writer[str]` em vez de `IO[str]`

`IO[str]` expõe toda a interface de arquivo; `dump_json` só usa `write()`; o protocolo mais enxuto pede só esse método

MÓDULO 1 · CLI.PY

CLI — TypedDict e deque

TYPEDDICT COM NOTAÇÃO DE FUNÇÃO

```python
# chave com hífen — notação de classe
# causaria SyntaxError
RawOperation = TypedDict(
    "RawOperation",
    {
        "operation": OperationKind,
        "unit-cost": float,
        "quantity": int,
    },
)
```

Por que notação de função?

A chave `unit-cost` tem hífen, por isso não pode virar atributo na sintaxe de classe

ALTERNATIVA CONSIDERADA: DEQUE E MAXLEN=0

```python
deque(
    map(
        partial(dump_json, output=writer_stream),
        map(
            process_operations_batch,
            map(
                parse_json_line,
                readlines(reader_stream)
            )
        ),
    ),
    maxlen=0,  # consome sem criar lista
)
```

Por que o for ficou?

Igualmente lazy (`readlines` é um gerador) e lê de cima para baixo, sem `deque` nem `partial`

Money — estrutura

```python
@dataclass(frozen=True)  # gera __init__, __eq__, __repr__
@total_ordering
class Money:
    amount: Decimal  # só Decimal, nunca float
    currency: str = DEFAULT_CURRENCY

    def __post_init__(self) -> None:
        # roda após o __init__ gerado pelo dataclass
        # frozen=True bloqueia setattr — hack necessário
        object.__setattr__(
            self, "amount",
            self.amount.quantize(TWOPLACES))

    @classmethod
    def zero(cls, currency=DEFAULT_CURRENCY) -> Money:
        return cls(Decimal("0.00"), currency)
```

01 **@dataclass gera __init__, __eq__, __repr__**

igualdade por valor e representação legível sem escrever boilerplate

02 **frozen=True: objeto imutável**

atribuir um campo depois da criação levanta FrozenInstanceError

03 **amount aceita apenas Decimal**

float não representa a maioria das frações decimais: `Decimal(1.005)` vira `1.00499…`; quem chama deixa a precisão explícita

04 **__post_init__ + object.__setattr__**

único lugar para arredondar para 2 casas decimais; contorna o bloqueio do frozen

05 **Money.zero(): construtor alternativo**

elimina repetição de `Money(Decimal("0.00"))`

Money — operações

```python
def __add__(self, other: Money) -> Money:
    if not isinstance(other, Money):
        return NotImplemented
    self._assert_same_currency_as(other)
    return Money(self.amount + other.amount, self.currency)

# __sub__ segue o padrão; __neg__ inverte o sinal
def __mul__(self, scalar: Scalar) -> Money:
    if not isinstance(scalar, SCALAR_TYPES):
        return NotImplemented
    return Money(self.amount * scalar, self.currency)

__rmul__ = __mul__  # __truediv__ segue o padrão
def __lt__(self, other: object) -> bool:
    if not isinstance(other, Money):
        return NotImplemented
    self._assert_same_currency_as(other)
    return self.amount < other.amount
```

01 **__add__ e __sub__: Money op Money**

retornam um novo objeto; operações entre moedas diferentes levantam `ValueError` via `_assert_same_currency_as`

02 **__mul__: Money × Scalar**

R$ 10 × 5 faz sentido; R$ 10 × R$ 10 não existe. `Scalar = int | Decimal`: `Money * 0.5` levanta TypeError em vez de passar por float

03 **__rmul__ = __mul__: comutatividade**

Python tenta `5 * money` via `int.__mul__` primeiro; ao falhar chama `money.__rmul__`, que reaproveita __mul__

04 **__lt__ + @total_ordering**

implementar só `__lt__` basta, porque o decorator deriva os demais; tipos estranhos retornam `NotImplemented`

Tax — estrutura

```python
@dataclass(frozen=True)
class InvestmentState:
    quantity: int = 0
    weighted_average_price: Money = field(
        default_factory=Money.zero)  # acumulador
    accumulated_loss: Money = field(
        default_factory=Money.zero)  # acumulador

@dataclass(frozen=True)
class OperationResult:
    new_state: InvestmentState
    tax: Money  # toda operação tem imposto (pode ser zero)

def process_operation(
    state: InvestmentState, operation: Operation
) -> OperationResult:
    match operation.kind:
        case "buy":  return handle_buy(state, operation)
        case "sell": return handle_sell(state, operation)
        case _ as unreachable: assert_never(unreachable)
```

01 InvestmentState: acumuladores com default_factory

`weighted_average_price` e `accumulated_loss` usam `Money.zero` como factory; começam em zero e acumulam ao longo das operações

02 OperationResult: estado e imposto juntos

toda operação retorna o novo estado e o imposto (que pode ser zero), então dá para auditar cada passo

03 Pattern matching com literais

match também desestrutura objetos e aceita guards; aqui ele só compara strings, como um `if/elif`; `assert_never` garante exaustividade em análise estática

Tax — handle_buy

```python
def handle_buy(
    state: InvestmentState,
    operation: Operation,
) → OperationResult:
    total_quantity = state.quantity + operation.quantity
    if total_quantity == 0:
        return OperationResult(new_state=state, tax=Money.zero())

    return OperationResult(
        new_state=InvestmentState(
            quantity=total_quantity,
            weighted_average_price=(
                (state.weighted_average_price * state.quantity)
                + (operation.unit_cost * operation.quantity)
            ) / total_quantity,
            accumulated_loss=state.accumulated_loss,
        ),
        tax=Money.zero(),
    )
```

01 Compra nunca gera imposto

`tax=Money.zero()` sempre, porque o imposto só incide em vendas com lucro

02 A aritmética é toda em Money

multiplicar quantidade (int) por preço (Money) retorna Money, sem escalares soltos na fórmula

03 accumulated_loss é preservado

compras não afetam o prejuízo acumulado, que passa intacto para o novo estado

04 Estado imutável

`InvestmentState` é frozen; handle_buy constrói e retorna um novo estado sem mutar o original

Tax — handle_sell

```
def calculate_tax(taxable, total) → Money:
    if taxable > Money.zero() and total > EXEMPTION_LIMIT:
        return taxable * TAX_RATE
    return Money.zero()

def handle_sell(state, op) → OperationResult:
    diff = op.unit_cost - state.weighted_average_price
    gross_profit = diff * op.quantity
    total = op.unit_cost * op.quantity

    net_profit = gross_profit - state.accumulated_loss
    taxable = max(Money.zero(), net_profit)
    new_loss = max(Money.zero(), -net_profit)

    return OperationResult(
        new_state=InvestmentState(...,
            accumulated_loss=new_loss),
        tax=calculate_tax(taxable, total),
    )
```

01 Lucro líquido calculado uma vez

lucro tributável e prejuízo a compensar são o mesmo valor com sinais opostos: `max(0, net)` e `max(0, -net)`, graças ao `__neg__`

02 calculate_tax no nível do módulo

as antigas funções internas não capturavam nada do escopo de handle_sell; como função de módulo, calculate_tax ganhou testes próprios

03 weighted_average_price não muda na venda

o preço médio ponderado só é recalculado em compras; vender não altera a base de custo

RETROSPECTIVA

Decisões técnicas

01 **Objeto Money como ponto de partida**

Padrão que eu já conhecia; garantiu precisão decimal desde o início, sem float

02 **Imutabilidade em todos os objetos**

frozen dataclasses eliminam bugs de estado compartilhado e tornam o fluxo previsível

03 **Resultado auditável por operação**

Retornar OperationResult com new_state permite inspecionar cada passo do cálculo

04 **Iterable como contrato de entrada**

Mais simples que IO; nos testes, uma lista de strings substitui o arquivo

05 **max(Money.zero(), ±net_profit)**

Um único lucro líquido substitui três condicionais

06 **Conteinerização + prek**

Ambiente e estilo de código fazem parte do entregável; o projeto roda igual em qualquer máquina



Demo

```
uv run capital-gains < entrada.txt
```



Dúvidas

capital-gains · Python · objeto dinheiro